

Computer Based Test (CBT) System

A robust, portable Computer Based Assessment application featuring dynamic test content loading, response storage, automatic evaluation, and cloud synchronisation (Dropbox integration).

System Requirements

Requirement	Details
OS	Windows 10 or later (64-bit)
Java 21+	Must be installed and available on the system PATH . Download from Oracle JDK 21 for Windows
Python 3.x	Either a portable interpreter (recommended) or a global installation.

Python Options

Option A – Portable Python (Recommended)

Download a portable Python package and extract it into a folder named **Python** in the project root so that **Python\python.exe** exists.

[🔗 Download Portable Python from SourceForge](#)

Option B – System Python

Install Python 3.x globally from [python.org](#). **start_test.bat** and **run_evaluator.bat** will detect it automatically if no portable interpreter is found.

Project Structure

```
Computer Based Test/
├─ start_test.bat           ← Entry point – run this to start
├─ run_evaluator.bat       ← Called automatically after the test
├─ load_data.py            ← Downloads assets and launches the Java GUI
├─ evaluator.py            ← Evaluates responses and uploads reports
├─ .env                    ← Environment configuration (not committed
to Git)
├─ dist/
│   └─ Computer_Based_Test.jar
├─ lib/                    ← Java dependency JARs
├─ src/
│   └─ Details.java
│   └─ Main.java           ← Java source
```

└─ Python/
to Git)

← Portable Python interpreter (not committed to Git)

Setup

1. Configure `.env`

Create a file named `.env` in the project root. This file is **never committed to Git**.

```
# — Hosted file URLs —————
CONFIG_LINK=https://raw.githubusercontent.com/imtiyaz-
allam/questions/refs/heads/main/config.json
QUESTION_LINK=https://raw.githubusercontent.com/imtiyaz-
allam/questions/refs/heads/main/question.json
SOLUTION_LINK=https://raw.githubusercontent.com/imtiyaz-
allam/questions/refs/heads/main/solution.json

# — Dropbox API credentials —————
DROPBOX_APP_KEY=your_app_key_here
DROPBOX_APP_SECRET=your_app_secret_here
DROPBOX_REFRESH_TOKEN=your_refresh_token_here

# — Local pre-exist flag —————
PRE_EXIST=false
```

`.env` Key Reference

Key	Type	Description
<code>CONFIG_LINK</code>	URL	Public URL of <code>config.json</code> . See config.json format below.
<code>QUESTION_LINK</code>	URL	Public URL of <code>question.json</code> , which maps subject names to CSV file URLs.
<code>SOLUTION_LINK</code>	URL	Public URL of the master answer-key <code>solution.json</code> used by the evaluator.
<code>DROPBOX_APP_KEY</code>	String	Dropbox OAuth2 App Key for automatic report uploads.
<code>DROPBOX_APP_SECRET</code>	String	Dropbox OAuth2 App Secret.
<code>DROPBOX_REFRESH_TOKEN</code>	String	Long-lived Dropbox refresh token.
<code>PRE_EXIST</code>	<code>true</code> / <code>false</code>	<code>true</code> → skip all downloads and run with existing local files. <code>false</code> (default) → download fresh copies of config, questions, and solution on every run.

[!NOTE]

Set `PRE_EXIST=true` when question and config files are already placed locally (e.g. running in an offline lab). Set to `false` for a standard cloud-hosted deployment.

config.json Format

`config.json` is a small JSON file hosted at the URL specified by `CONFIG_LINK` in `.env` (or hardcoded in `Main.java` as `CONFIG_URL`). It is downloaded automatically at startup and saved locally as `C:\CBT\config.json`.

Minimum Required Format

```
{
  "dur": 180,
  "solution": "https://example.com/path/to/solution.json"
}
```

Field Reference

Field	Type	Required	Description
<code>dur</code>	Integer	Yes	Exam duration in minutes . Converted to seconds internally. Defaults to <code>180</code> if missing.
<code>solution</code>	String (URL)	No	Public URL to download <code>solution.json</code> after the test ends. Can be overridden by <code>SOLUTION_LINK</code> in <code>.env</code> .

Full Example

```
{
  "dur": 120,
  "solution": "https://raw.githubusercontent.com/your-org/your-repo/main/solution.json"
}
```

[!IMPORTANT]

The `solution` URL in `config.json` acts as a **fallback**. If `SOLUTION_LINK` is set in `.env`, it takes priority over the value in `config.json`.

[!NOTE]

System Requirement: Windows. The application is designed to run on Windows.

`start_test.bat` and `run_evaluator.bat` are Windows batch scripts. Java and Python must be accessible on the system `PATH` or via the portable interpreter placed in the `Python\` folder.

Running

Standard Test Mode

Double-click `start_test.bat` or run it from a terminal:

```
start_test.bat
```

What happens:

1. Java installation is verified; an error with the download link is shown if missing.
2. Python interpreter is located (portable first, then system).
3. `load_data.py` runs:
 - If `PRE_EXIST=true`: downloads `config.json`, question CSVs, and `solution.json` into the correct directories, then launches the Java GUI.
 - If `PRE_EXIST=false`: skips downloads and launches the Java GUI directly.
4. Student fills in their details, takes the test, and submits.
5. Java saves responses and automatically calls `run_evaluator.bat`.
6. `evaluator.py` scores the responses, generates a CSV report, and uploads it to Dropbox.

Admin Mode (Answer-Key Creation)

```
start_test.bat --admin-mode
```

What happens:

1. Same Python and Java checks as above.
2. Downloads `config.json` and question CSVs **only** (solution download is skipped — the admin creates it).
3. Launches the Java GUI **without** the student details dialog.
4. Admin navigates through all questions selecting the correct answers.
5. On test end, the selected answers are saved as `solutions/solution.json` and `solution.json`.
6. `evaluator.py` is **not** launched.

Installing Python Packages

Both `load_data.py` and `evaluator.py` perform a startup dependency check and print a single install command if any package is missing.

`evaluator.py` requires the **Dropbox SDK**:

```
python -m pip install dropbox
```

Or, using the portable interpreter:

```
Python\python.exe -m pip install dropbox
```

If you extend `load_data.py` with additional libraries (e.g. `boto3`, `requests`), add them to the `_REQUIRED_PACKAGES` list at the top of the file — users will automatically receive the correct install command.

Customising Downloads for Secure Cloud Storage

If your files are hosted on a private/authenticated storage service (AWS S3, Azure Blob, Google Cloud Storage, or any API-key-protected server), modify the download helpers in `load_data.py` to add the required headers.

Example — Adding Authentication Headers

Replace the plain `urllib.request.urlopen(url)` pattern in any download function with:

```
def download_secure_file(url, target_path, headers=None):
    """Generic authenticated downloader. Pass a dict of HTTP headers."""
    import urllib.request
    print(f"Downloading {target_path} from {url} ...")
    try:
        req = urllib.request.Request(url)

        # — Add your auth headers here —————
        if headers:
            for key, value in headers.items():
                req.add_header(key, value)
        # —————

        with urllib.request.urlopen(req, timeout=15) as resp:
            data = resp.read()

        os.makedirs(os.path.dirname(target_path) or ".", exist_ok=True)
        with open(target_path, "wb") as f_out:
            f_out.write(data)
        print(f"Saved to {target_path}")
        return True
    except Exception as e:
        print(f"Download failed: {e}")
        return False
```

AWS S3 / API token example

```
download_secure_file(  
    url="https://your-bucket.s3.amazonaws.com/config.json",  
    target_path="config.json",  
    headers={  
        "Authorization": "Bearer YOUR_TOKEN",  
        "x-api-key": "YOUR_API_KEY",  
    }  
)
```

Google Cloud Storage (signed URL — no headers needed)

```
download_secure_file(  
    url="https://storage.googleapis.com/your-bucket/config.json?X-Goog-Signature=...",  
    target_path="config.json",  
)
```

Store any secrets used here in `.env` (add new keys as needed) and read them via `env.get("YOUR_KEY")` — **never hard-code credentials in source files.**

.gitignore Summary

The following are excluded from version control:

Excluded	Reason
<code>.env</code>	Contains secrets / personal credentials
<code>Python/</code>	Large binary; user installs their own
<code>__pycache__/</code>	Python bytecode cache
<code>config.json</code> , <code>qstns/</code> , <code>solutions/</code> , <code>response/</code>	Downloaded or generated at runtime
<code>examinee_details.json</code>	Personal test data
<code>build/</code> , <code>nbproject/private/</code>	IDE / build artefacts